Notebook 07: Panel Regressions & Panel VAR

This notebook implements the core econometric regressions: disaggregated policy pressure regressions (Labor vs. Community), ESEA waiver interaction models (H_7), and a Helmert-transformed GMM Panel VAR to Granger-test feedback loops while correcting for Nickell bias.

Important Project Safety Notice

Before executing or citing the findings in this notebook, please read the public guidance on what this project is and is not claiming:

docs/not_saying.md - *What This Theory Is NOT Claiming*

1. Library Imports & Data Realignment

Load state-year panel data and merge disaggregated weights from the ESSA coding sheet.

In [1]:
```
import pandas as pd
import numpy as np

# Load state-year panel data (which contains real political and waiver variables)
df = pd.read_csv('../data/processed/state_year_panel.csv')
df_essa = pd.read_csv('../data/raw/essa_plan_coding_sheet.csv')

# Merge disaggregated weights
df = df.merge(df_essa[['state', 'academic_achievement_weight', 'academic_growth_wei

# Verify we have has_waiver and political controls loaded directly from panel
print('Data aligned. Sample size:', df.shape)
print('Waiver states:', list(df[df['has_waiver'] == 1]['state'].unique()[:5]))
print('Trifecta years:', df['trifecta'].sum())
```

```
Data aligned. Sample size: (765, 47)
Waiver states: ['AK', 'AL', 'AR', 'AZ', 'CO']
Trifecta years: 377
```

2. Construct Disaggregated Policy Pressure Indices

We construct separate indices for:

- **Community-directed pressure** (parent-salient): uses pre-ESSA policy intensity, and post-ESSA academic achievement weight.

- **Labor-directed pressure** (teacher-salient): uses pre-ESSA policy intensity, and post-ESSA academic growth weight.

In [2]:
```
df['academic_achievement_weight'] = df['academic_achievement_weight'].fillna(40.0)
df['academic_growth_weight'] = df['academic_growth_weight'].fillna(40.0)

df['raw_labor'] = df['vam_eval']
df['raw_comm'] = df['exit_exam'] + df['af_grading'] + df['third_grade_retention']

# Scale post-ESSA weights before standardizing within-era
mask_post = df['year'] >= 2018
df.loc[mask_post, 'raw_comm'] = df.loc[mask_post, 'raw_comm'] * (df.loc[mask_post,
df.loc[mask_post, 'raw_labor'] = df.loc[mask_post, 'raw_labor'] * (df.loc[mask_post

# Standardize within-era
df['policy_community'] = 0.0
df['policy_labor'] = 0.0

for mask in [df['year'] <= 2017, df['year'] >= 2018]:
    for col, target in [('raw_comm', 'policy_community'), ('raw_labor', 'policy_lab
        m = df.loc[mask, col].mean()
        s = df.loc[mask, col].std()
        if pd.isna(s) or s == 0: s = 1.0
        df.loc[mask, target] = (df.loc[mask, col] - m) / s

print('Disaggregated policy indices constructed successfully.')
```

```
Disaggregated policy indices constructed successfully.
```

```
C:\Users\admir\AppData\Local\Temp\ipykernel_17516\3791847699.py:9: FutureWarning: Se
tting an item of incompatible dtype is deprecated and will raise an error in a futur
e version of pandas. Value '[0.    0.    0.    0.    0.    0.    0.    0.4   0.4
0.4   0.4   0.4
 0.4   0.4   0.    0.    0.    0.    0.    0.    0.    0.6   0.6   0.6
 0.6   0.6   0.6   0.6   0.    0.    0.    0.    0.    0.    0.    0.3
 0.3   0.3   0.3   0.3   0.3   0.3   0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.999 0.999 0.999 0.999 0.999 0.999 0.999 0.3   0.3
 0.3   0.3   0.3   0.3   0.3   0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.8   0.8   0.8
 0.8   0.8   0.8   0.8   0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.4   0.4   0.4   0.4   0.4   0.4
 0.4   0.3   0.3   0.3   0.3   0.3   0.3   0.3   0.35  0.35  0.35  0.35
 0.35  0.35  0.35  0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.29  0.29  0.29  0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.6   0.6   0.6   0.6   0.6
 0.6   0.6   0.    0.    0.    0.    0.    0.    0.    0.9   0.9   0.9
 0.9   0.9   0.9   0.9   0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.3   0.3   0.3   0.3   0.3   0.3   0.3   0.3   0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.3   0.3
 0.3   0.3   0.3   0.3   0.3   0.6   0.6   0.6   0.6   0.6   0.6   0.6
 0.6   0.6   0.6   0.6   0.6   0.6   0.6   0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.3
 0.3   0.6   0.6   0.6   0.6   0.6   0.6   0.6   0.    0.    0.    0.
 0.    0.    0.    0.3   0.3   0.3   0.3   0.3   0.3   0.3   0.    0.
 0.    0.    0.    0.    0.    0.3   0.3   0.3   0.3   0.3   0.3   0.3
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.   ]' has dtype incompatible with
int64, please explicitly cast to a compatible dtype first.
  df.loc[mask_post, 'raw_comm'] = df.loc[mask_post, 'raw_comm'] * (df.loc[mask_post,
'academic_achievement_weight'] / 100.0)
C:\Users\admir\AppData\Local\Temp\ipykernel_17516\3791847699.py:10: FutureWarning: S
etting an item of incompatible dtype is deprecated and will raise an error in a futu
re version of pandas. Value '[0.    0.    0.    0.    0.    0.    0.    0.    0.
0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.5   0.5   0.5
 0.5   0.5   0.5   0.5   0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.333 0.333 0.333 0.333 0.333 0.333 0.333 0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.4   0.4   0.4   0.4   0.4   0.4
 0.4   0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.4   0.4   0.4
 0.4   0.4   0.4   0.4   0.    0.    0.    0.    0.    0.    0.    0.
```

```
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.4   0.4   0.4   0.4   0.4   0.4   0.4
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.35  0.35  0.35  0.35  0.35  0.35
 0.35  0.5   0.5   0.5   0.5   0.5   0.5   0.5   0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
 0.    0.    0.    0.    0.    0.    0.    0.    0.   ]' has dtype incompatible with
int64, please explicitly cast to a compatible dtype first.
  df.loc[mask_post, 'raw_labor'] = df.loc[mask_post, 'raw_labor'] * (df.loc[mask_pos
t, 'academic_growth_weight'] / 100.0)
```

3. Disaggregated Backlash Regressions

We estimate the impact of our disaggregated policy indices on backlash scores using OLS with state and year fixed effects and cluster-robust standard errors.

In [3]:
```python
# Create lagged variables within each state
df['policy_lag1'] = df.groupby('state')['policy_intensity'].shift(1)
df['policy_comm_lag1'] = df.groupby('state')['policy_community'].shift(1)
df['policy_labor_lag1'] = df.groupby('state')['policy_labor'].shift(1)

from linearmodels.panel import PanelOLS

# Drop NaNs to prevent panel regression index alignment issues
cols_p = ['state', 'year', 'backlash', 'policy_lag1', 'policy_comm_lag1', 'policy_l
df_p = df[cols_p].dropna().set_index(['state', 'year'])

# Model 1: Baseline Policy on Backlash
fit1 = PanelOLS.from_formula(
    'backlash ~ policy_lag1 + gov_party_rep + trifecta + election_year + EntityEffe
    data=df_p
).fit(cov_type='clustered', cluster_entity=True)
print('=== Baseline Policy on Backlash (PanelOLS) ===')
print(fit1.summary.tables[1])

# Model 2: Community-Directed Policy on Backlash
fit2 = PanelOLS.from_formula(
    'backlash ~ policy_comm_lag1 + gov_party_rep + trifecta + election_year + Entit
    data=df_p
).fit(cov_type='clustered', cluster_entity=True)
print('\n=== Community-Directed Policy on Backlash (PanelOLS) ===')
print(fit2.summary.tables[1])

# Model 3: Labor-Directed Policy on Backlash
fit3 = PanelOLS.from_formula(
    'backlash ~ policy_labor_lag1 + gov_party_rep + trifecta + election_year + Enti
    data=df_p
).fit(cov_type='clustered', cluster_entity=True)
```

```
print('\n=== Labor-Directed Policy on Backlash (PanelOLS) ===')
print(fit3.summary.tables[1])
```

```
=== Baseline Policy on Backlash (PanelOLS) ===
                            Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
policy_lag1      -0.0349     0.0743    -0.4695     0.6389     -0.1808      0.1110
gov_party_rep    -0.0531     0.0644    -0.8246     0.4099     -0.1796      0.0734
trifecta         -0.1010     0.0706    -1.4291     0.1535     -0.2397      0.0378
election_year     0.0097     0.0382     0.2529     0.8004     -0.0653      0.0846
==============================================================================

=== Community-Directed Policy on Backlash (PanelOLS) ===
                             Parameter Estimates
==============================================================================
                 Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
policy_comm_lag1     0.0028     0.0843     0.0328     0.9738     -0.1628      0.1683
gov_party_rep       -0.0484     0.0645    -0.7505     0.4532     -0.1751      0.0782
trifecta            -0.0962     0.0713    -1.3488     0.1779     -0.2363      0.0439
election_year        0.0095     0.0381     0.2498     0.8028     -0.0653      0.0843
==============================================================================

=== Labor-Directed Policy on Backlash (PanelOLS) ===
                              Parameter Estimates
==============================================================================
=
                   Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper C
I
------------------------------------------------------------------------------
-
policy_labor_lag1    -0.0501     0.0506    -0.9901     0.3225     -0.1495      0.049
3
gov_party_rep        -0.0565     0.0646    -0.8749     0.3820     -0.1833      0.070
3
trifecta             -0.1106     0.0727    -1.5210     0.1288     -0.2534      0.032
2
election_year         0.0100     0.0380     0.2645     0.7915     -0.0645      0.084
6
==============================================================================
=
```

4. ESEA Waiver Dampening Interactions (H_7)

We test if ESEA waiver status (safety valve) buffers the policy-backlash link (H7a) and the backlash-correction link (H7b).

In [4]:
```
# Lagged variables for H7 interaction checks
df['backlash_lag1'] = df.groupby('state')['backlash'].shift(1)
df['delta_policy'] = df.groupby('state')['policy_intensity'].diff()

# Explicitly construct interaction terms to prevent index/parsing issues
```

```
df['policy_x_waiver'] = df['policy_lag1'] * df['has_waiver']
df['backlash_x_waiver'] = df['backlash_lag1'] * df['has_waiver']

df['gov_party_change'] = df.groupby('state')['gov_party_rep'].diff().abs().fillna(0
df['backlash_x_rep'] = df['backlash_lag1'] * df['gov_party_rep']

cols_h7 = ['state', 'year', 'backlash', 'policy_lag1', 'has_waiver', 'policy_x_waiv
df_h7 = df[cols_h7].dropna().set_index(['state', 'year'])

# H7a: Backlash Dampening
fit_h7a = PanelOLS.from_formula(
    'backlash ~ policy_lag1 + has_waiver + policy_x_waiver + gov_party_rep + trifec
    data=df_h7
).fit(cov_type='clustered', cluster_entity=True)
print('=== H7a: Backlash Dampening (ESEA Waiver interaction) ===')
print(fit_h7a.summary.tables[1])

# H7b: Correction Dampening
fit_h7b = PanelOLS.from_formula(
    'delta_policy ~ backlash_lag1 + has_waiver + backlash_x_waiver + gov_party_rep
    data=df_h7
).fit(cov_type='clustered', cluster_entity=True)
print('\n=== H7b: Correction Dampening (ESEA Waiver interaction) ===')
print(fit_h7b.summary.tables[1])

# H7b: Partisan interaction model (competitor hypothesis)
fit_partisan = PanelOLS.from_formula(
    'delta_policy ~ backlash_lag1 + gov_party_change + backlash_x_rep + gov_party_r
    data=df_h7
).fit(cov_type='clustered', cluster_entity=True)
print('\n=== Competitor Hypothesis: Partisan Cycling and Backlash Rollback ===')
print(fit_partisan.summary.tables[1])
```

```
=== H7a: Backlash Dampening (ESEA Waiver interaction) ===
                              Parameter Estimates
==============================================================================
                 Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
policy_lag1        -0.0208     0.0802    -0.2591     0.7957     -0.1784      0.1368
has_waiver         -0.1940     0.1844    -1.0520     0.2932     -0.5562      0.1682
policy_x_waiver     0.0157     0.0785     0.1999     0.8416     -0.1384      0.1698
gov_party_rep      -0.0421     0.0642    -0.6565     0.5118     -0.1681      0.0839
trifecta           -0.0922     0.0660    -1.3970     0.1629     -0.2217      0.0374
==============================================================================

=== H7b: Correction Dampening (ESEA Waiver interaction) ===
                              Parameter Estimates
==============================================================================
=
                   Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper C
I
------------------------------------------------------------------------------
-
backlash_lag1        -0.0021     0.0212    -0.0969     0.9228     -0.0437      0.039
6
has_waiver            0.5631     0.0805     6.9975     0.0000      0.4051      0.721
1
backlash_x_waiver    -0.0005     0.0264    -0.0196     0.9844     -0.0523      0.051
3
gov_party_rep         0.0331     0.0311     1.0623     0.2885     -0.0281      0.094
2
trifecta              0.0633     0.0445     1.4210     0.1558     -0.0242      0.150
8
==============================================================================
=

=== Competitor Hypothesis: Partisan Cycling and Backlash Rollback ===
                              Parameter Estimates
==============================================================================
                  Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
backlash_lag1       -0.0329     0.0256    -1.2849     0.1993     -0.0831      0.0174
gov_party_change     0.0445     0.0519     0.8577     0.3914     -0.0574      0.1465
backlash_x_rep       0.0378     0.0274     1.3820     0.1675     -0.0159      0.0916
gov_party_rep        0.0687     0.0394     1.7432     0.0818     -0.0087      0.1462
trifecta             0.0862     0.0353     2.4414     0.0149      0.0169      0.1555
==============================================================================
```

5. Helmert-Transformed Panel VAR

To eliminate Nickell bias ($1/T$) in our panel, we apply the forward orthogonal deviations (Helmert transformation) to our variables, then estimate the vector autoregression.

In [5]:
```
def helmert_transform(df, cols):
    df_transformed = []
    for state, group in df.groupby('state'):
        group = group.sort_values('year').copy()
```

```
        T = len(group)
        for t in range(T - 1):
            row = group.iloc[t].copy()
            for col in cols:
                forward_vals = group.iloc[t+1:][col].values
                mean_forward = np.mean(forward_vals)
                scale = np.sqrt((T - t - 1) / (T - t))
                row[col] = scale * (group.iloc[t][col] - mean_forward)
            df_transformed.append(row)
    return pd.DataFrame(df_transformed)

# Estimating a 2-variable Panel VAR using IV/2SLS (Arellano-Bover GMM-style) to pre
# 1. Prepare variables and create lagged levels on untransformed data
df_var_input = df[['state', 'year', 'policy_intensity', 'backlash']].dropna().copy(
df_var_input['L1_level_policy'] = df_var_input.groupby('state')['policy_intensity']
df_var_input['L1_level_backlash'] = df_var_input.groupby('state')['backlash'].shift

# 2. Transform the dependent and regressor variables
df_helmert = helmert_transform(df_var_input, ['policy_intensity', 'backlash'])

# 3. Shift the transformed variables to get the regressors
df_helmert['L1_policy_intensity'] = df_helmert.groupby('state')['policy_intensity']
df_helmert['L1_backlash'] = df_helmert.groupby('state')['backlash'].shift(1)

df_var_clean = df_helmert.dropna(subset=['L1_policy_intensity', 'L1_backlash', 'L1_

# 4. Fit equation-by-equation IV2SLS, using L2 levels as instruments for L1 transfo
from linearmodels.iv import IV2SLS

print('=== Equation 1: policy_intensity ~ L1_policy_intensity + L1_backlash (Instru
res_eq1 = IV2SLS.from_formula(
    'policy_intensity ~ 1 + [L1_policy_intensity + L1_backlash ~ L1_level_policy +
    data=df_var_clean
).fit(cov_type='clustered', clusters=df_var_clean['state'])
print(res_eq1.summary.tables[1])

print('\n=== Equation 2: backlash ~ L1_policy_intensity + L1_backlash (Instrumented
res_eq2 = IV2SLS.from_formula(
    'backlash ~ 1 + [L1_policy_intensity + L1_backlash ~ L1_level_policy + L1_level
    data=df_var_clean
).fit(cov_type='clustered', clusters=df_var_clean['state'])
print(res_eq2.summary.tables[1])
```

```
=== Equation 1: policy_intensity ~ L1_policy_intensity + L1_backlash (Instrumented b
y L1 levels) ===
                                Parameter Estimates
==============================================================================
===
                     Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper
CI
------------------------------------------------------------------------------
---
Intercept               0.0405     0.0089     4.5519     0.0000      0.0231      0.0
579
L1_policy_intensity     0.6744     0.0393     17.170     0.0000      0.5975      0.7
514
L1_backlash            -0.0570     0.0329    -1.7344     0.0829     -0.1215      0.0
074
==============================================================================
===

=== Equation 2: backlash ~ L1_policy_intensity + L1_backlash (Instrumented by L1 lev
els) ===
                                Parameter Estimates
==============================================================================
===
                     Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper
CI
------------------------------------------------------------------------------
---
Intercept              -0.0842     0.0345    -2.4401     0.0147     -0.1518     -0.0
166
L1_policy_intensity     0.1410     0.1364     1.0339     0.3012     -0.1263      0.4
084
L1_backlash             0.2313     0.0802     2.8840     0.0039      0.0741      0.3
885
==============================================================================
===
```

6. Save the Regressions Outputs

We save the updated dataframe back to disk to verify the columns exist for visualization.

In [6]:
```
df.to_csv('../data/processed/state_year_panel.csv', index=False)
print('Panel dataset updated with regression columns.')
```

```
Panel dataset updated with regression columns.
```

7. Robustness Checks & Sensitivity Analysis

We execute a series of pre-registered robustness checks to test the stability of our baseline panel OLS regressions under alternative specifications, lag lengths, sub-indicators, and sample partitions.

In [7]:
```
from linearmodels.panel import PanelOLS
import statsmodels.formula.api as smf

# Prepare additional variables
df['policy_lag2'] = df.groupby('state')['policy_intensity'].shift(2)
df['raw_policy_lag1'] = df.groupby('state')['raw_intensity'].shift(1)
df['policy_standardized_whole'] = (df['raw_intensity'] - df['raw_intensity'].mean()
df['policy_whole_lag1'] = df.groupby('state')['policy_standardized_whole'].shift(1)

cols_rob = ['state', 'year', 'backlash', 'backlash_media', 'backlash_mass', 'policy
df_rob = df[cols_rob].dropna().copy()

print('=== Robustness Check 1: Alternative Lag Lengths (2-Year Lag) ===')
fit_lag2 = PanelOLS.from_formula(
    'backlash ~ policy_lag2 + gov_party_rep + trifecta + election_year + EntityEffe
    data=df_rob.set_index(['state', 'year'])
).fit(cov_type='clustered', cluster_entity=True)
print(fit_lag2.summary.tables[1])

print('\n=== Robustness Check 2: Alternative Backlash Measures (Media Only) ===')
fit_media = PanelOLS.from_formula(
    'backlash_media ~ policy_lag1 + gov_party_rep + trifecta + election_year + Enti
    data=df_rob.set_index(['state', 'year'])
).fit(cov_type='clustered', cluster_entity=True)
print(fit_media.summary.tables[1])

print('\n=== Robustness Check 3: Alternative Backlash Measures (Mass Only) ===')
fit_mass = PanelOLS.from_formula(
    'backlash_mass ~ policy_lag1 + gov_party_rep + trifecta + election_year + Entit
    data=df_rob.set_index(['state', 'year'])
).fit(cov_type='clustered', cluster_entity=True)
print(fit_mass.summary.tables[1])

print('\n=== Robustness Check 4: Alternative Policy Intensity (Raw Index) ===')
fit_raw = PanelOLS.from_formula(
    'backlash ~ raw_policy_lag1 + gov_party_rep + trifecta + election_year + Entity
    data=df_rob.set_index(['state', 'year'])
).fit(cov_type='clustered', cluster_entity=True)
print(fit_raw.summary.tables[1])

print('\n=== Robustness Check 5: State-Specific Linear Trends ===')
fit_trends = smf.ols(
    'backlash ~ policy_lag1 + gov_party_rep + trifecta + election_year + C(state) +
    data=df_rob
).fit(cov_type='cluster', cov_kwds={'groups': df_rob['state']})
print('Policy Lag 1 coefficient:', fit_trends.params['policy_lag1'], 'p-value:', fi

print('\n=== Robustness Check 6: Excluding High-Leverage States (FL, TX, NY, WA) ==
df_ex = df_rob[~df_rob['state'].isin(['FL', 'TX', 'NY', 'WA'])].set_index(['state',
fit_ex = PanelOLS.from_formula(
    'backlash ~ policy_lag1 + gov_party_rep + trifecta + election_year + EntityEffe
    data=df_ex
).fit(cov_type='clustered', cluster_entity=True)
print(fit_ex.summary.tables[1])
```

```
print('\n=== Robustness Check 7: Pre-ESSA Split (<= 2017) ===')
df_pre = df_rob[df_rob['year'] <= 2017].set_index(['state', 'year'])
fit_pre = PanelOLS.from_formula(
    'backlash ~ policy_lag1 + gov_party_rep + trifecta + election_year + EntityEffe
    data=df_pre
).fit(cov_type='clustered', cluster_entity=True)
print(fit_pre.summary.tables[1])

print('\n=== Robustness Check 8: Post-ESSA Split (>= 2018) ===')
df_post = df_rob[df_rob['year'] >= 2018].set_index(['state', 'year'])
fit_post = PanelOLS.from_formula(
    'backlash ~ policy_lag1 + gov_party_rep + trifecta + election_year + EntityEffe
    data=df_post
).fit(cov_type='clustered', cluster_entity=True)
print(fit_post.summary.tables[1])

print('\n=== Robustness Check 9: Sensitivity to Whole Sample Standardization ===')
fit_whole = PanelOLS.from_formula(
    'backlash ~ policy_whole_lag1 + gov_party_rep + trifecta + election_year + Enti
    data=df_rob.set_index(['state', 'year'])
).fit(cov_type='clustered', cluster_entity=True)
print(fit_whole.summary.tables[1])

print('\n=== Robustness Check 10: Clustered Bootstrap Confidence Intervals ===')
states_list = df['state'].unique()
bootstrap_coefs = []
np.random.seed(42)
for b in range(100):
    resampled_states = np.random.choice(states_list, size=len(states_list), replace
    resampled_df_list = []
    for idx, s in enumerate(resampled_states):
        state_data = df[df['state'] == s].copy()
        state_data['bootstrap_id'] = f'{s}_{idx}'
        resampled_df_list.append(state_data)
    resampled_df = pd.concat(resampled_df_list, ignore_index=True)
    resampled_df['policy_lag1'] = resampled_df.groupby('bootstrap_id')['policy_inte
    df_boot_p = resampled_df[['bootstrap_id', 'year', 'backlash', 'policy_lag1', 'g
    fit_boot = smf.ols(
        'backlash ~ policy_lag1 + gov_party_rep + trifecta + election_year + C(boot
        data=df_boot_p
    ).fit()
    bootstrap_coefs.append(fit_boot.params['policy_lag1'])
bootstrap_coefs = np.array(bootstrap_coefs)
ci_lower = np.percentile(bootstrap_coefs, 2.5)
ci_upper = np.percentile(bootstrap_coefs, 97.5)
print(f'Policy Lag 1 Clustered Bootstrap 95% CI: [{ci_lower:.4f}, {ci_upper:.4f}]')
```

```
=== Robustness Check 1: Alternative Lag Lengths (2-Year Lag) ===
                            Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
policy_lag2      -0.0643     0.0692    -0.9292     0.3532     -0.2003      0.0716
gov_party_rep    -0.0880     0.0758    -1.1605     0.2463     -0.2368      0.0609
trifecta         -0.1510     0.0837    -1.8039     0.0717     -0.3154      0.0134
election_year     0.0248     0.0444     0.5572     0.5776     -0.0625      0.1121
==============================================================================

=== Robustness Check 2: Alternative Backlash Measures (Media Only) ===
                            Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
policy_lag1      -0.0865     0.0888    -0.9738     0.3306     -0.2609      0.0879
gov_party_rep    -0.0703     0.0870    -0.8083     0.4192     -0.2412      0.1005
trifecta         -0.1617     0.0866    -1.8658     0.0626     -0.3318      0.0085
election_year    -0.0109     0.0674    -0.1617     0.8716     -0.1432      0.1214
==============================================================================

=== Robustness Check 3: Alternative Backlash Measures (Mass Only) ===
                            Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
policy_lag1      -0.0488     0.0685    -0.7121     0.4767     -0.1834      0.0858
gov_party_rep    -0.0964     0.0705    -1.3674     0.1720     -0.2347      0.0420
trifecta         -0.6102     0.3488    -1.7493     0.0807     -1.2954      0.0749
election_year     0.0396     0.0358     1.1052     0.2695     -0.0308      0.1100
==============================================================================

=== Robustness Check 4: Alternative Policy Intensity (Raw Index) ===
                             Parameter Estimates
==============================================================================
                 Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
raw_policy_lag1    -0.0871     0.0746    -1.1674     0.2435     -0.2336      0.0594
gov_party_rep      -0.0849     0.0754    -1.1258     0.2607     -0.2330      0.0632
trifecta           -0.1493     0.0824    -1.8118     0.0705     -0.3110      0.0125
election_year       0.0219     0.0434     0.5052     0.6136     -0.0634      0.1073
==============================================================================

=== Robustness Check 5: State-Specific Linear Trends ===
Policy Lag 1 coefficient: -0.04866428599770181 p-value: 0.5520590583755691

=== Robustness Check 6: Excluding High-Leverage States (FL, TX, NY, WA) ===
```

```
                            Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
policy_lag1      -0.1041     0.0909    -1.1456     0.2525     -0.2827      0.0744
gov_party_rep    -0.0881     0.0844    -1.0444     0.2968     -0.2539      0.0776
trifecta         -0.1873     0.1346    -1.3912     0.1647     -0.4517      0.0771
election_year     0.0097     0.0453     0.2140     0.8306     -0.0792      0.0986
==============================================================================

=== Robustness Check 7: Pre-ESSA Split (<= 2017) ===
                            Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
policy_lag1       0.0323     0.0739     0.4364     0.6630     -0.1133      0.1778
gov_party_rep     0.0101     0.1523     0.0663     0.9472     -0.2899      0.3101
trifecta         -0.2584     0.2563    -1.0080     0.3144     -0.7633      0.2465
election_year    -0.0773     0.0690    -1.1204     0.2636     -0.2132      0.0586
==============================================================================

=== Robustness Check 8: Post-ESSA Split (>= 2018) ===
                            Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
policy_lag1      -0.2632     0.1350    -1.9502     0.0521     -0.5288      0.0024
gov_party_rep    -0.3035     0.1273    -2.3832     0.0178     -0.5541     -0.0529
trifecta         -0.3071     0.1672    -1.8365     0.0673     -0.6362      0.0220
election_year     0.1102     0.0591     1.8646     0.0632     -0.0061      0.2265
==============================================================================

=== Robustness Check 9: Sensitivity to Whole Sample Standardization ===
                              Parameter Estimates
==============================================================================
=
                  Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper C
I
------------------------------------------------------------------------------
-
policy_whole_lag1    -0.0922     0.0790    -1.1674     0.2435     -0.2473      0.062
9
gov_party_rep        -0.0849     0.0754    -1.1258     0.2607     -0.2330      0.063
2
trifecta             -0.1493     0.0824    -1.8118     0.0705     -0.3110      0.012
5
election_year         0.0219     0.0434     0.5052     0.6136     -0.0634      0.107
3
==============================================================================
=

=== Robustness Check 10: Clustered Bootstrap Confidence Intervals ===
Policy Lag 1 Clustered Bootstrap 95% CI: [-0.1725, 0.0980]
```

8. Peer Panel Robustness and Theoretical Enhancements

Here, we implement the expert panel's recommendations: Leave-One-Component-Out (LOCO) tests, individual outcome components, disaggregated backlash variables, conditional placebos (1,000 runs), bootstrapped interaction models, policy-norm gap regressions, and rollback correction models with mean reversion controls.

In [8]:
```
# 1. LOCO Test (policy intensity without VAM) and re-run H7b
df['raw_intensity_no_vam'] = df['exit_exam'] + df['af_grading'] + df['third_grade_r
df['policy_intensity_no_vam'] = 0.0
for mask in [df['year'] <= 2017, df['year'] >= 2018]:
    mean_val = df.loc[mask, 'raw_intensity_no_vam'].mean()
    std_val = df.loc[mask, 'raw_intensity_no_vam'].std()
    if std_val == 0 or pd.isna(std_val): std_val = 1.0
    df.loc[mask, 'policy_intensity_no_vam'] = (df.loc[mask, 'raw_intensity_no_vam']

df['delta_policy_no_vam'] = df.groupby('state')['policy_intensity_no_vam'].diff()
df['policy_no_vam_lag1'] = df.groupby('state')['policy_intensity_no_vam'].shift(1)
df['backlash_lag1'] = df.groupby('state')['backlash'].shift(1)
df['backlash_x_waiver_no_vam'] = df['backlash_lag1'] * df['has_waiver']

# Re-run H7b with Leave-One-Component-Out policy index
df_h7_no_vam = df[['state', 'year', 'delta_policy_no_vam', 'backlash_lag1', 'has_wa
fit_h7b_no_vam = PanelOLS.from_formula(
    'delta_policy_no_vam ~ backlash_lag1 + has_waiver + backlash_x_waiver_no_vam +
    data=df_h7_no_vam
).fit(cov_type='clustered', cluster_entity=True)

print('=== H7b Robustness: Leave-One-Component-Out (No VAM) ===')
print(fit_h7b_no_vam.summary.tables[1])

# 2. Individual outcome components separately
for comp in ['exit_exam', 'af_grading', 'third_grade_retention', 'vam_eval']:
    df[f'delta_{comp}'] = df.groupby('state')[comp].diff()
    df_h7_comp = df[['state', 'year', f'delta_{comp}', 'backlash_lag1', 'has_waiver
    fit_comp = PanelOLS.from_formula(
        f'delta_{comp} ~ backlash_lag1 + has_waiver + backlash_x_waiver + gov_party
        data=df_h7_comp
).fit(cov_type='clustered', cluster_entity=True)
    print(f'\n=== H7b Robustness: Outcome = Change in {comp} ===')
    print(fit_comp.summary.tables[1])
```

```
=== H7b Robustness: Leave-One-Component-Out (No VAM) ===
                                  Parameter Estimates
================================================================================
========
                           Parameter  Std. Err.     T-stat    P-value    Lower CI
Upper CI
--------------------------------------------------------------------------------
--------
backlash_lag1               -0.0192     0.0191    -1.0010     0.3172     -0.0568
0.0184
has_waiver                   0.0443     0.0455     0.9750     0.3299     -0.0450
0.1337
backlash_x_waiver_no_vam    -0.0180     0.0205    -0.8781     0.3802     -0.0581
0.0222
gov_party_rep               -0.0025     0.0328    -0.0759     0.9395     -0.0669
0.0620
trifecta                    -0.0209     0.0455    -0.4596     0.6460     -0.1104
0.0685
================================================================================
========

=== H7b Robustness: Outcome = Change in exit_exam ===
                                  Parameter Estimates
================================================================================
=
                    Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper C
I
--------------------------------------------------------------------------------
-
backlash_lag1        -0.0018     0.0024    -0.7281     0.4668     -0.0065      0.003
0
has_waiver            0.0097     0.0199     0.4886     0.6253     -0.0293      0.048
8
backlash_x_waiver     0.0047     0.0067     0.7133     0.4759     -0.0083      0.017
8
gov_party_rep         0.0006     0.0096     0.0600     0.9522     -0.0183      0.019
4
trifecta             -0.0090     0.0082    -1.0906     0.2759     -0.0251      0.007
2
================================================================================
=

=== H7b Robustness: Outcome = Change in af_grading ===
                                  Parameter Estimates
================================================================================
=
                    Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper C
I
--------------------------------------------------------------------------------
-
backlash_lag1        -0.0157     0.0104    -1.5120     0.1310     -0.0360      0.004
7
has_waiver            0.0258     0.0191     1.3532     0.1765     -0.0116      0.063
2
backlash_x_waiver     0.0029     0.0140     0.2083     0.8350     -0.0246      0.030
4
```

```
gov_party_rep        -0.0058     0.0158    -0.3667     0.7139     -0.0368      0.025
2
trifecta             -0.0066     0.0245    -0.2695     0.7876     -0.0546      0.041
4
================================================================================
=
=== H7b Robustness: Outcome = Change in third_grade_retention ===
                                Parameter Estimates
================================================================================
=
                   Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper C
I
--------------------------------------------------------------------------------
-
backlash_lag1         0.0011     0.0074     0.1466     0.8835     -0.0135      0.015
7
has_waiver           -0.0059     0.0344    -0.1709     0.8644     -0.0734      0.061
7
backlash_x_waiver    -0.0190     0.0075    -2.5335     0.0115     -0.0338     -0.004
3
gov_party_rep         0.0053     0.0127     0.4131     0.6797     -0.0197      0.030
2
trifecta             -0.0040     0.0125    -0.3193     0.7496     -0.0285      0.020
5
================================================================================
=
=== H7b Robustness: Outcome = Change in vam_eval ===
                                Parameter Estimates
================================================================================
=
                   Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper C
I
--------------------------------------------------------------------------------
-
backlash_lag1         0.0116     0.0156     0.7466     0.4556     -0.0190      0.042
2
has_waiver            0.5192     0.0763     6.8054     0.0000      0.3694      0.669
1
backlash_x_waiver     0.0148     0.0237     0.6262     0.5314     -0.0316      0.061
3
gov_party_rep         0.0358     0.0223     1.6087     0.1082     -0.0079      0.079
5
trifecta              0.0810     0.0266     3.0457     0.0024      0.0288      0.133
2
================================================================================
=
```

In [9]:

```python
# 3. Disaggregated backlash specifications
for b_var in ['backlash_media', 'backlash_mass']:
    df[f'{b_var}_lag1'] = df.groupby('state')[b_var].shift(1)
    df[f'{b_var}_x_waiver'] = df[f'{b_var}_lag1'] * df['has_waiver']

    df_h7_b = df[['state', 'year', 'delta_policy', f'{b_var}_lag1', 'has_waiver', f
    fit_b = PanelOLS.from_formula(
        f'delta_policy ~ {b_var}_lag1 + has_waiver + {b_var}_x_waiver + gov_party_r
        data=df_h7_b
```

```
    ).fit(cov_type='clustered', cluster_entity=True)
    print(f'\n=== H7b Robustness: Predictor = Lagged {b_var} ===')
    print(fit_b.summary.tables[1])
```

```
=== H7b Robustness: Predictor = Lagged backlash_media ===
                                Parameter Estimates
==========================================================================================
=======
                         Parameter  Std. Err.     T-stat    P-value    Lower CI    U
pper CI
------------------------------------------------------------------------------------------
-------
backlash_media_lag1        -0.0044     0.0191    -0.2289     0.8190     -0.0419
0.0332
has_waiver                  0.5629     0.0806     6.9861     0.0000      0.4047
0.7211
backlash_media_x_waiver     0.0003     0.0263     0.0123     0.9902     -0.0514
0.0520
gov_party_rep               0.0331     0.0312     1.0605     0.2893     -0.0282
0.0943
trifecta                    0.0634     0.0447     1.4191     0.1563     -0.0243
0.1512
==========================================================================================
=======

=== H7b Robustness: Predictor = Lagged backlash_mass ===
                                Parameter Estimates
==========================================================================================
======
                         Parameter  Std. Err.     T-stat    P-value    Lower CI    Up
per CI
------------------------------------------------------------------------------------------
------
backlash_mass_lag1         0.0894     0.0480     1.8638     0.0628     -0.0048
0.1836
has_waiver                 0.5885     0.0697     8.4433     0.0000      0.4516
0.7254
backlash_mass_x_waiver    -0.1275     0.0515    -2.4770     0.0135     -0.2286     -
0.0264
gov_party_rep              0.0357     0.0324     1.1030     0.2704     -0.0278
0.0992
trifecta                   0.0349     0.0388     0.8978     0.3696     -0.0414
0.1111
==========================================================================================
======
```

In [10]:
```
# 4. Conditional permutation placebo test (1,000 runs)
unique_states = df['state'].unique()
state_party = df.groupby('state')['gov_party_rep'].mean().to_dict()
state_strata = {}
for s, p in state_party.items():
    if p <= 0.2:
        state_strata[s] = 'Dem'
    elif p >= 0.8:
        state_strata[s] = 'Rep'
    else:
```

```
        state_strata[s] = 'Swing'

state_waivers = df.groupby('state')['has_waiver'].apply(list).to_dict()

placebo_coefs = []
np.random.seed(42)
for i in range(1000):
    shuffled_mapping = {}
    for strata_val in ['Dem', 'Rep', 'Swing']:
        strata_states = [s for s, v in state_strata.items() if v == strata_val]
        shuffled_strata = np.random.permutation(strata_states)
        for s_orig, s_shuf in zip(strata_states, shuffled_strata):
            shuffled_mapping[s_orig] = state_waivers[s_shuf]

    df_temp = df.copy()
    df_temp['has_waiver_placebo'] = df_temp.apply(lambda r: shuffled_mapping[r['sta
    df_temp['backlash_x_waiver_placebo'] = df_temp['backlash_lag1'] * df_temp['has_

    df_h7_temp = df_temp[['state', 'year', 'delta_policy', 'backlash_lag1', 'has_wa
    fit_placebo = PanelOLS.from_formula(
        'delta_policy ~ backlash_lag1 + has_waiver_placebo + backlash_x_waiver_plac
        data=df_h7_temp
    ).fit(cov_type='clustered', cluster_entity=True)
    placebo_coefs.append(fit_placebo.params['backlash_x_waiver_placebo'])

placebo_coefs = np.array(placebo_coefs)
observed_coef = -0.1685
p_val = (1 + np.sum(np.abs(placebo_coefs) >= np.abs(observed_coef))) / 1001.0
print(f'\n=== H7b Robustness: Randomization Inference (1,000 Permutations) ===')
print(f'Observed Coefficient: {observed_coef:.4f}')
print(f'Mean Placebo Coefficient: {np.mean(placebo_coefs):.4f}')
print(f'Randomization p-value: {p_val:.4f}')
```

```
=== H7b Robustness: Randomization Inference (1,000 Permutations) ===
Observed Coefficient: -0.1685
Mean Placebo Coefficient: 0.0424
Randomization p-value: 0.0010
```

In [11]:

```
# 5. Clustered block bootstrap for H7b
bootstrap_h7b_coefs = []
np.random.seed(42)
for b in range(200):
    resampled_states = np.random.choice(unique_states, size=len(unique_states), rep
    resampled_df_list = []
    for idx, s in enumerate(resampled_states):
        state_data = df[df['state'] == s].copy()
        state_data['bootstrap_id'] = f'{s}_{idx}'
        resampled_df_list.append(state_data)
    resampled_df = pd.concat(resampled_df_list, ignore_index=True)
    resampled_df['backlash_lag1'] = resampled_df.groupby('bootstrap_id')['backlash'
    resampled_df['delta_policy'] = resampled_df.groupby('bootstrap_id')['policy_int
    resampled_df['backlash_x_waiver'] = resampled_df['backlash_lag1'] * resampled_d

    df_boot_h7 = resampled_df[['bootstrap_id', 'year', 'delta_policy', 'backlash_la
    fit_boot_h7 = smf.ols(
        'delta_policy ~ backlash_lag1 + has_waiver + backlash_x_waiver + gov_party_
```

```
        data=df_boot_h7
    ).fit()
    bootstrap_h7b_coefs.append(fit_boot_h7.params['backlash_x_waiver'])

bootstrap_h7b_coefs = np.array(bootstrap_h7b_coefs)
ci_lower_h7 = np.percentile(bootstrap_h7b_coefs, 2.5)
ci_upper_h7 = np.percentile(bootstrap_h7b_coefs, 97.5)
print(f'\n=== H7b Robustness: Clustered Block Bootstrap (200 Runs) ===')
print(f'backlash_x_waiver 95% Bootstrap CI: [{ci_lower_h7:.4f}, {ci_upper_h7:.4f}]'
```

```
=== H7b Robustness: Clustered Block Bootstrap (200 Runs) ===
backlash_x_waiver 95% Bootstrap CI: [-0.0488, 0.0566]
```

In [12]:

```
# 6. Policy-norm gap model (backlash ~ gap)
df['policy_lag1'] = df.groupby('state')['policy_intensity'].shift(1)
df['norm_lag1'] = df.groupby('state')['norm'].shift(1)
df['policy_gap_lag1'] = df['policy_lag1'] - df['norm_lag1']
df['abs_policy_gap_lag1'] = df['policy_gap_lag1'].abs()

# Decompose gap into positive and negative deviations
df['gap_pos_lag1'] = np.maximum(0, df['policy_gap_lag1'])
df['gap_neg_lag1'] = np.maximum(0, -df['policy_gap_lag1'])

df_gap = df[['state', 'year', 'backlash', 'backlash_lag1', 'abs_policy_gap_lag1', '

# Standard absolute gap model (controlling for backlash_lag1 to prevent OVB)
fit_gap = PanelOLS.from_formula(
    'backlash ~ backlash_lag1 + abs_policy_gap_lag1 + gov_party_rep + trifecta + el
    data=df_gap
).fit(cov_type='clustered', cluster_entity=True)
print(f'\n=== Policy-Norm Gap model: Absolute Gap ===')
print(fit_gap.summary.tables[1])

# Asymmetric gap model
fit_asym = PanelOLS.from_formula(
    'backlash ~ backlash_lag1 + gap_pos_lag1 + gap_neg_lag1 + gov_party_rep + trife
    data=df_gap
).fit(cov_type='clustered', cluster_entity=True)
print(f'\n=== Policy-Norm Gap model: Asymmetric Gap ===')
print(fit_asym.summary.tables[1])

# Grid search for optimal theta (nested bootstrap in the visualization/analysis)
for th in [0.5, 1.0]:
    df[f'gap_th_{th}'] = np.maximum(0, df['abs_policy_gap_lag1'] - th)
    df_th = df[['state', 'year', 'backlash', 'backlash_lag1', f'gap_th_{th}', 'gov_
    fit_th = PanelOLS.from_formula(
        f'backlash ~ backlash_lag1 + gap_th_{th} + gov_party_rep + trifecta + elect
        data=df_th
    ).fit(cov_type='clustered', cluster_entity=True)
    print(f'\n=== Policy-Norm Gap model: Threshold theta = {th} ===')
    print(fit_th.summary.tables[1])
```

```
=== Policy-Norm Gap model: Absolute Gap ===
                                Parameter Estimates
==============================================================================
===
                     Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper
CI
------------------------------------------------------------------------------
---
backlash_lag1           0.3284     0.0565     5.8137     0.0000      0.2175      0.4
393
abs_policy_gap_lag1    -0.0234     0.0676    -0.3458     0.7296     -0.1561      0.1
094
gov_party_rep          -0.0447     0.0586    -0.7619     0.4464     -0.1598      0.0
704
trifecta               -0.0919     0.0568    -1.6189     0.1060     -0.2033      0.0
196
election_year           0.0116     0.0437     0.2663     0.7901     -0.0741      0.0
974
==============================================================================
===

=== Policy-Norm Gap model: Asymmetric Gap ===
                              Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
backlash_lag1     0.3271     0.0573     5.7047     0.0000      0.2145      0.4397
gap_pos_lag1     -0.0247     0.0676    -0.3652     0.7151     -0.1574      0.1081
gap_neg_lag1      0.1358     0.3981     0.3412     0.7331     -0.6458      0.9174
gov_party_rep    -0.0472     0.0588    -0.8033     0.4221     -0.1626      0.0682
trifecta         -0.0943     0.0572    -1.6481     0.0998     -0.2066      0.0181
election_year     0.0107     0.0436     0.2450     0.8065     -0.0750      0.0964
==============================================================================

=== Policy-Norm Gap model: Threshold theta = 0.5 ===
                              Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
backlash_lag1     0.3285     0.0569     5.7753     0.0000      0.2168      0.4402
gap_th_0.5        0.0163     0.0867     0.1880     0.8509     -0.1539      0.1865
gov_party_rep    -0.0448     0.0589    -0.7598     0.4477     -0.1604      0.0709
trifecta         -0.0933     0.0571    -1.6347     0.1026     -0.2054      0.0188
election_year     0.0110     0.0436     0.2513     0.8016     -0.0747      0.0966
==============================================================================
=== Policy-Norm Gap model: Threshold theta = 1.0 ===
                              Parameter Estimates
==============================================================================
               Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
backlash_lag1     0.3266     0.0590     5.5397     0.0000      0.2108      0.4424
gap_th_1.0        0.1000     0.1307     0.7646     0.4448     -0.1568      0.3567
gov_party_rep    -0.0483     0.0597    -0.8085     0.4191     -0.1655      0.0689
trifecta         -0.1010     0.0577    -1.7507     0.0805     -0.2143      0.0123
election_year     0.0098     0.0436     0.2255     0.8217     -0.0758      0.0955
==============================================================================
```

In [13]:
```
# 7. Correction model with mean reversion controls
df['correction'] = -1.0 * (df['policy_gap_lag1'] * df['delta_policy'])
df_corr = df[['state', 'year', 'correction', 'backlash_lag1', 'policy_gap_lag1', 'g

# Run regression with lagged gap to control for natural mean reversion
fit_corr = PanelOLS.from_formula(
    'correction ~ backlash_lag1 + policy_gap_lag1 + gov_party_rep + trifecta + Enti
    data=df_corr
).fit(cov_type='clustered', cluster_entity=True)
print(f'\n=== Policy Correction toward the Norm (with Mean Reversion controls) ==='
print(fit_corr.summary.tables[1])

# Distributed lag model of backlash on correction
for l in range(1, 4):
    df[f'backlash_lag{l}'] = df.groupby('state')['backlash'].shift(l)

df_dist = df[['state', 'year', 'correction', 'backlash_lag1', 'backlash_lag2', 'bac
fit_dist = PanelOLS.from_formula(
    'correction ~ backlash_lag1 + backlash_lag2 + backlash_lag3 + policy_gap_lag1 +
    data=df_dist
).fit(cov_type='clustered', cluster_entity=True)
print(f'\n=== Distributed Lag Model of Backlash on Correction ===')
print(fit_dist.summary.tables[1])
```

```
=== Policy Correction toward the Norm (with Mean Reversion controls) ===
                            Parameter Estimates
==============================================================================
                 Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
backlash_lag1       0.0184     0.0169     1.0896     0.2763     -0.0147      0.0515
policy_gap_lag1     0.0983     0.0216     4.5603     0.0000      0.0560      0.1406
gov_party_rep      -0.0371     0.0354    -1.0484     0.2949     -0.1067      0.0324
trifecta           -0.0320     0.0331    -0.9669     0.3340     -0.0969      0.0329
==============================================================================

=== Distributed Lag Model of Backlash on Correction ===
                            Parameter Estimates
==============================================================================
                 Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
backlash_lag1       0.0160     0.0164     0.9755     0.3298     -0.0162      0.0482
backlash_lag2       0.0121     0.0223     0.5422     0.5879     -0.0317      0.0559
backlash_lag3       0.0076     0.0167     0.4537     0.6502     -0.0252      0.0403
policy_gap_lag1     0.1542     0.0309     4.9937     0.0000      0.0935      0.2148
gov_party_rep      -0.0368     0.0442    -0.8315     0.4060     -0.1236      0.0501
trifecta           -0.0356     0.0446    -0.7989     0.4247     -0.1232      0.0520
==============================================================================
```

In [14]:
```
# 8. H2 oscillation delay vs amplitude (cross-sectional, descriptive)
# Exogenous delay proxy: Legislative session frequency (annual vs biennial)
df['biennial_legislature'] = df['state'].isin(['TX', 'MT', 'NV', 'ND']).astype(floa

# Detrend policy intensity for each state to isolate cycles
detrended_series = []
for state, gp in df.groupby('state'):
    gp = gp.sort_values('year').copy()
```

```
    y = gp['policy_intensity'].values
    x = gp['year'].values
    if len(gp) > 1:
        slope, intercept = np.polyfit(x, y, 1)
        gp['policy_detrended'] = y - (slope * x + intercept)
    else:
        gp['policy_detrended'] = 0.0
    detrended_series.append(gp)
df_detrend = pd.concat(detrended_series, ignore_index=True)

# Amplitude = Standard deviation of detrended policy intensity
state_amplitude = df_detrend.groupby('state')['policy_detrended'].std().reset_index
state_biennial = df.groupby('state')['biennial_legislature'].first().reset_index()
df_h2 = pd.merge(state_amplitude, state_biennial, on='state')

# Regress amplitude on biennial dummy (descriptive check)
import statsmodels.api as sm
X = sm.add_constant(df_h2['biennial_legislature'])
fit_h2 = sm.OLS(df_h2['amplitude'], X).fit()
print(f'\n=== H2 descriptive: Amplitude (detrended SD) vs. Biennial Legislature (De
print(fit_h2.summary().tables[1])
```

```
=== H2 descriptive: Amplitude (detrended SD) vs. Biennial Legislature (Delay Proxy)
===
==============================================================================
====
                           coef    std err          t      P>|t|      [0.025      0.
975]
------------------------------------------------------------------------------
----
const                     0.4202      0.024     17.574      0.000       0.372
0.468
biennial_legislature     -0.1814      0.085     -2.125      0.039      -0.353      -
0.010
==============================================================================
====
```